

PROGRAMACIÓN CONCURRENTE Y PARALELA

Práctica 1: Modelo de cómputo y Exclusión Mutua

1er Cuatrimestre 2026

Ejercicio 1. Para los siguientes programas realice lo siguiente asumiendo que las operaciones atómicas corresponden a la lectura y asignación de variables, operaciones aritméticas y evaluación de expresiones booleanas:

- Dar la semántica de los hilos T1, T2 y su ejecución concurrente T1|T2 (en función de un estado σ , ignorando la declaración e inicialización de las variables globales).
- Dar el diagrama de transición de estados completo de T1|T2 a partir del estado inicial provisto.

a) **global** x = 1
global y = 2

thread T1	thread T2
y = x	x = y

b) **global** x = 0
global y = 0

thread T1	thread T2
y = x + 1	x = y + 1

c) **global** x = 0
global y = 0

thread T1	thread T2
while (x<1)	x = 1
y=y+1	

Ejercicio 2. Dado el siguiente programa, para algún K fijo,

global n=0

thread T1	thread T2
do K times	do K times
n = n + 1	n = n + 1

- Escriba al menos dos trazas del programa en las que el valor final de n sea $2K$
- Escriba al menos una traza del programa en la que el valor final de n sea K .
- ¿Es posible que al final de la ejecución del programa el valor final de n sea menor a K ?

Ejercicio 3. Considere el siguiente programa concurrente:

```

global x = 1
global y = 2
global z = 3

        thread T1  thread T2  thread T3
            y = x      z = y      x = z
    
```

¿Cuáles son los posibles valores finales para x , y , y z ?

Ejercicio 4. Sea P un programa concurrente que consiste de N threads. Cada uno de estos threads está compuesto por K instrucciones atómicas y, por simplicidad, asumimos que no contienen bucles ni otras estructuras de control (es decir, todo thread requiere ejecutar exactamente K pasos para terminar).

- Proporcione una cota inferior para la cantidad de nodos del diagrama de transición de estados de P en función de N y K . ¿En qué caso esta cota no sería ajustada?
- ¿Cuál es la cantidad total de trazas posibles para la ejecución de P ?

Ejercicio 5. Asumir que la función f tiene una raíz entera, es decir, $f(x) = 0$ para algún valor x entero. A continuación proponemos distintos programas para encontrar tal raíz. Consideraremos a un programa correcto si ambos threads terminan cuando uno de ellos ha encontrado la raíz. Para cada programa, decir si es correcto o no, justificando la respuesta.

a) Programa A

```

global found

thread T1 {
    local i = 0
    found = false
    while (!found) {
        i = i + 1
        found = (f(i) == 0)
    }
}

thread T2 {
    local j = 1
    found = false
    while (!found){
        j = j - 1
        found = (f(j) == 0)
    }
}
    
```

b) Programa B

```

global found = false

thread T1 {
    local i = 0
    while (!found) {
        i = i + 1
        found = (f(i) == 0)
    }
}

thread T2 {
    local j = 1
    while (!found) {
        j = j - 1
        found = (f(j) == 0)
    }
}
    
```

c) Programa C

```

global found = false

thread T1 {
    local i = 0
    while (!found) {
        i = i + 1
        if (f(i) == 0)
            found = true
    }
}

thread T2 {
    local j = 1;
    while (!found) {
        j = j - 1;
        if (f(j) == 0)
            found = true
    }
}
    
```

Ejercicio 6. Considerar el siguiente programa:

```

global n = 0

thread T1
    while (n < 2)
        print (n)

thread T2 {
    n = n + 1;
    n = n + 1;
}
    
```

- ¿Cuántas veces puede aparecer 2 en la salida?
- ¿Cuántas veces puede aparecer 1 en la salida?
- ¿Cuál es la longitud de la secuencia mas corta que puede ser mostrada?

Ejercicio 7. Considerar el siguiente programa:

```

global n = 0

thread T1
    while (n < 1)
        n = n + 1

thread T2
    while (n >= 0)
        n = n - 1
    
```

- ¿Existe un *interleaving* en el que el loop en T1 ejecute exactamente una vez? Justificar.
- ¿Existe un *interleaving* en el que el loop en T1 no termina?.

Ejercicio 8. Considerar el siguiente programa:

```

global n = 0
global flag = false

thread
    while (!flag)
        n = 1 - n

thread
    while (!flag)
        if n == 0
            flag = true
    
```

- a) ¿Cuáles son los posibles valores de n cuando el programa termina? Justificar.
- b) ¿Puede una ejecución del programa no terminar?

Ejercicio 9. Considerar la siguiente versión del algoritmo de *bakery* para dos threads:

```

global np = 0
global nq = 0

thread p
    while(true){
        np = nq + 1
        while (nq != 0
                && np > nq){}
        // seccion critica
        np = 0
    }

thread q
    while(true){
        nq = np + 1
        while (np != 0
                && nq > np){}
        // seccion critica
        nq = 0
    }
    
```

¿Resuelve este algoritmo el problema de la exclusión mutua? Justificar.

Ejercicio 10. Considerar la siguiente propuesta para resolver el problema de exclusión mutua para n threads:

```

global actual = 0
global turnos = 0

PedirTurno(){
    local turno = turnos
    turnos = turnos + 1
    return turno
}

LiberarTurno(){
    actual = actual + 1
    turnos = turnos - 1
}
    
```

Considere, también, que cada thread ejecuta el siguiente protocolo:

```

//SECCION NO CRITICA
local miturno = PedirTurno()
while (actual != miturno){}
//SECCION CRITICA
LiberarTurno()
//SECCION NO CRITICA
    
```

- a) Mostrar que esta propuesta no resuelve el problema de la exclusión mutua. Indicar claramente cuál/es condición/es son violadas e ilustrar mostrando una traza.

b) ¿Qué sucede si `PedirTurno` y `LiberarTurno` son operaciones atómicas?

Ejercicio 11. Considerar la siguiente extensión del algoritmo de Peterson para n procesos (con $n > 2$) que utiliza las siguientes variables compartidas:

```
global flag[n] = {false, false, ..., false}
global turno = 0
```

Además, cada thread está identificado por el valor de la variable local `id` (que toma valores entre 0 y $n - 1$). Cada thread utiliza el siguiente protocolo.

```
thread(id) {
    //SECCION NO CRITICA
    flag[id] = true;
    local otro = (id + 1) % n
    turno = otro
    while (flag[otro] && turno == otro){}
    //SECCION CRITICA
    flag[id] = false
    //SECCION NO CRITICA
}
```

Mostrar que esta variación no resuelve el problema de la exclusión mutua para n procesos, con $n > 2$. Indique claramente cuál/es condición/es se cumplen y cuales no justificando apropiadamente en cada caso.

Ejercicio 12. Considerar la siguiente propuesta para resolver el problema de la exclusión mutua para n procesos (con $n > 2$) que utiliza las siguientes variables compartidas:

```
global flag[n] = {false, false, ..., false};
```

y la siguiente función auxiliar:

```
algunVerdadero(id) {
    local aux = false;
    for (int i = 0; i < n; i++) {
        if (i != id)
            aux = aux || flag[i]
    }
    return aux;
}
```

Además cada thread está identificado por el valor de la variable local `id` (que toma valores entre 0 y $n - 1$). Cada thread utiliza el siguiente protocolo.

```
thread(id) {
    //SECCION NO CRITICA
    flag[id] = true;
    while (!algunVerdadero(id)){
    //SECCION CRITICA
```

```

    flag[id] = false;
    //SECCION NO CRITICA
}

```

- a) Mostrar que esta propuesta no resuelve el problema de la exclusión mutua para n procesos con $n > 2$ si la operación `algunVerdadero` no es atómica. Indique claramente cuál/es condición/es se cumplen y cuales no justificando apropiadamente en cada caso.
- b) ¿Qué sucede si `algunVerdadero` es una operación atómica?

Ejercicio 13. Dado el algoritmo de *bakery*, mostrar que la condición $j < id$ en el segundo `while` es necesaria. Es decir, muestre que el algoritmo que se obtiene al eliminar esta condición no resuelve el problema de la exclusión mutua. Mencionar al menos una propiedad que viola el algoritmo obtenido y mostrar una traza de ejecución que lo evidencie. ¿Hay alguna propiedad que siga valiendo?

Por comodidad, damos a continuación el algoritmo modificado (donde se eliminó la condición $j < id$):

```

global entrando[N] = {false, ..., false}
global numero[N] = {0, ..., 0}

thread(id) {
    //SECCION NO CRITICA
    entrando[id] = true
    numero[id] = 1 + max(numero[0], ..., numero[n-1])
    entrando[id] = false

    for (j = 0; j < n; j++) {
        while (entrando[j]){}
        while (numero[j] != 0 && (numero[j] <= numero[id] ||
                                   (numero[j] == numero[id] &&
                                    j != id))){}
    }
    //SECCION CRITICA
    numero[id] = 0
    //SECCION NO CRITICA
}

```

Ejercicio 14. Considere la operación atómica `fetch-and-add` definida de la siguiente manera (las variables parámetro se pasan por referencia):

```

fetch-and-add(compartida, propia, x) {
    propia = compartida
    compartida = compartida + x
}

```

y el siguiente algoritmo para resolver el problema de la exclusión mutua que utiliza las variables compartidas `ticket` y `turno`, ambas inicializadas en 0.

```
//SECCION NO CRITICA
local miturno
fetch-and-add(ticket, miturno, 1)
while (turno != miturno) {}
//SECCION CRITICA
fetch-and-add(ticket, miturno, -1)
//SECCION NO CRITICA
```

Explique por qué esta implementación no resuelve el problema de la exclusión mutua; modifíquela para que sí lo haga y argumente la correctitud de su solución.

Ejercicio 15. Considere la siguiente operación:

```
tomarFlag(mia, otro) {
    flag[mia] = !flag[otro]
}
```

Se propone el siguiente algoritmo para resolver el problema de la exclusión mutua entre dos threads que utilizan el array compartido `flag`:

```
global flag[0..1]={false,false}

thread T0 {
    while (!flag[0])
        tomarFlag(0,1)
    //SECCION CRITICA
    flag[0]=false
}

thread T1 {
    while (!flag[1])
        tomarFlag(1,0)
    //SECCION CRITICA
    flag[1]=false
}
```

Se solicita:

- Asumiendo que la operación `tomarFlag` no es atómica, ¿resuelve la propuesta anterior el problema de la exclusión mutua? Justifique su respuesta.
- En el caso en que `tomarFlag` es una operación atómica, el algoritmo es una solución al problema de la exclusión mutua. Justifique por qué.